

Heterogeneous data analysis and string similarity

Anthony Kamau

1 Heterogeneous data analysis & string similarity

In this lab, you will practice to compute the similarity between different strings and sets. You will use different atomic similarity measures. Moreover, you will practice to find the similarity between different documents using min-hash and locality sensitive hashing beside the traditional ones (Jaccard similarity, Sørensen Coefficient, Tversky Index and the Overlap Coefficient)

2 Atomic String Similarity

In the following exercises, you will learn about finding the similarity between atomic values using Edit-Distance, Jaro Similarity, Jaro-Winkler Similarity and Soundex). We will start by importing the required libraries.

```
[1]: import pandas as pd
import csv
import numpy as np
```

2.1 Minimum Edit Distance (Levenshtein)

```
[2]: '''
TODO: find an implementation of the Levenshtein minimum edit distance and use it
to find the similarity between INTENTION and EXECUTION
'''
def ldist(s, t):
    """
    iterative levenshtein(s, t) -> ldist
    ldist is the Levenshtein distance between the strings
    s and t.
    For all i and j, dist[i,j] will contain the Levenshtein
    distance between the first i characters of s and the
    first j characters of t
    """

    rows = len(s)+1
    cols = len(t)+1
    dist = [[0 for x in range(cols)] for x in range(rows)]
```

```

# source prefixes can be transformed into empty strings
# by deletions:
for i in range(1, rows):
    dist[i][0] = i

# target prefixes can be created from an empty source string
# by inserting the characters
for i in range(1, cols):
    dist[0][i] = i

for col in range(1, cols):
    for row in range(1, rows):
        if s[row-1] == t[col-1]:
            c = 0
        else:
            c = 2
        dist[row][col] = min(dist[row-1][col] + 1,      # deletion
                             dist[row][col-1] + 1,      # insertion
                             dist[row-1][col-1] + c)    # substitution

for r in range(rows):
    print(dist[r])

return dist[row][col]

print("Minimum edit distance = ", ldist("INTENTION", "EXECUTION"))

```

```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 6, 7, 8]
[2, 3, 4, 5, 6, 7, 8, 7, 8, 7]
[3, 4, 5, 6, 7, 8, 7, 8, 9, 8]
[4, 3, 4, 5, 6, 7, 8, 9, 10, 9]
[5, 4, 5, 6, 7, 8, 9, 10, 11, 10]
[6, 5, 6, 7, 8, 9, 8, 9, 10, 11]
[7, 6, 7, 8, 9, 10, 9, 8, 9, 10]
[8, 7, 8, 9, 10, 11, 10, 9, 8, 9]
[9, 8, 9, 10, 11, 12, 11, 10, 9, 8]
Minimum edit distance = 8

```

```

[3]: '''
      TODO: Modify the ldist function to accept a new parameter which specifies the
            ↪substitution cost
      Test your implementation with the call mod_ldist("Intention", "execution", 1)
            ↪-- You should get 5
      and mod_ldist("Intention", "execution", 2) -- You should get 8

```

```

'''
def mod_ldist(s, t, substitution_cost):
    """
        iterative_levenshtein(s, t) -> ldist
        ldist is the Levenshtein distance between the strings
        s and t.
        For all i and j, dist[i,j] will contain the Levenshtein
        distance between the first i characters of s and the
        first j characters of t
    """

    rows = len(s)+1
    cols = len(t)+1
    dist = [[0 for x in range(cols)] for x in range(rows)]

    # source prefixes can be transformed into empty strings
    # by deletions:
    for i in range(1, rows):
        dist[i][0] = i

    # target prefixes can be created from an empty source string
    # by inserting the characters
    for i in range(1, cols):
        dist[0][i] = i

    for col in range(1, cols):
        for row in range(1, rows):
            if s[row-1] == t[col-1]:
                c = 0
            else:
                c = substitution_cost
            dist[row][col] = min(dist[row-1][col] + 1,          # deletion
                                dist[row][col-1] + 1,          # insertion
                                dist[row-1][col-1] + c)         # substitution

    return dist[row][col]

print("Minimum edit distance = ", mod_ldist("INTENTION", "EXECUTION", 2))

```

Minimum edit distance = 8

2.2 Jaro Similarity

```

[4]: '''
    TODO: find an implementation of the Jaro similarity algorithm and use it
    to find the similarity between the following pairs of strings:
    ('nichlseon', 'niechlson')

```

```

('MARTHA', 'MARHTA'),
('DIXON', 'DICKSONX'),
('JELLYFISH', 'SMELLYFISH'),
('cunningham', 'cuningham'),
('shackleford', 'shackelford'),
('nichleson', 'nichulson'),
('arnab', 'urban')
('DATASCIENCE', 'SCIENCEDATA')
'''
def jaro(s, t):
    '''Jaro similarity between two strings.'''
    s_len = len(s)
    t_len = len(t)

    if s_len == 0 and t_len == 0:
        return 1

    match_distance = (max(s_len, t_len) // 2) - 1

    s_matches = [False] * s_len
    t_matches = [False] * t_len

    matches = 0
    transpositions = 0

    for i in range(s_len):
        start = max(0, i - match_distance)
        end = min(i + match_distance + 1, t_len)

        for j in range(start, end):
            if t_matches[j]:
                continue
            if s[i] != t[j]:
                continue
            s_matches[i] = True
            t_matches[j] = True
            matches += 1
            break

    if matches == 0:
        return 0

    k = 0
    for i in range(s_len):
        if not s_matches[i]:
            continue
        while not t_matches[k]:

```

```

        k += 1
    if s[i] != t[k]:
        transpositions += 1
    k += 1

return ((matches / s_len) +
        (matches / t_len) +
        ((matches - transpositions / 2) / matches)) / 3

```

2.3 Jaro-Winkler Similarity

```

[5]: '''
    TODO: find an implementation of the Jaro-Winkler similarity algorithm and use it
    to find the similarity between the pairs of strings in the previous exercise.
    Compare the results and comment on them.
    '''

def jaro_winkler(s, t, p = 0.1):
    # Compute the Jaro similarity
    jaro_sim = jaro(s, t)
    ls = len(s)
    lt = len(t)
    # Initialize the upper bound for the no. of prefixes.
    min_l = min(ls, lt)

    # Compute the prefix matches.
    l = 0
    for i in range(min_l):
        if s[i] == t[i]:
            l += 1
        else:
            break
    # Return the similarity value.
    L = min(l, 4)
    return (jaro_sim + ( L * p * (1 - jaro_sim) ))

```

Now we test the two methods on the given pairs of strings.

Example

```

[6]: for s, t in [('nichlseon', 'niechlson'),
                 ('MARTHA', 'MARHTA'),
                 ('DIXON', 'DICKSONX'),
                 ('JELLYFISH', 'SMELLYFISH'),
                 ('cunningham', 'cunnigham'),
                 ('shackleford', 'shackelford'),
                 ('nichleson', 'nichulson'),
                 ('arnab', 'urban'),
                 ('DATASCIENCE', 'SCIENCEDATA')]

```

```

]:
    print("jaro(%r, %r) = %.4f and jaro-winkler(%r, %r) = %.4f" % (s, t,
↪jaro(s, t), s, t, jaro_winkler(s, t)))

```

```

jaro('nichlseon', 'niechlson') = 0.9259 and jaro-winkler('nichlseon',
'niechlson') = 0.9407
jaro('MARTHA', 'MARHTA') = 0.9444 and jaro-winkler('MARTHA', 'MARHTA') = 0.9611
jaro('DIXON', 'DICKSONX') = 0.7667 and jaro-winkler('DIXON', 'DICKSONX') =
0.8133
jaro('JELLYFISH', 'SMELLYFISH') = 0.8963 and jaro-winkler('JELLYFISH',
'SMELLYFISH') = 0.8963
jaro('cunningham', 'cunnigham') = 0.9667 and jaro-winkler('cunningham',
'cunnigham') = 0.9800
jaro('shackleford', 'shackelford') = 0.9697 and jaro-winkler('shackleford',
'shackelford') = 0.9818
jaro('nichleson', 'nichulson') = 0.9259 and jaro-winkler('nichleson',
'nichulson') = 0.9556
jaro('arnab', 'urban') = 0.6000 and jaro-winkler('arnab', 'urban') = 0.6000
jaro('DATASCIENCE', 'SCIENCEDATA') = 0.7576 and jaro-winkler('DATASCIENCE',
'SCIENCEDATA') = 0.7576

```

Soundex

```

[7]: '''
TODO: find an implementation of the soundex algorithm and use it
to find the representation of EDUCATE and EVALUATE
'''
def soundex(name):

    """
    The Soundex algorithm assigns a 1-letter + 3-digit code to strings,
    the intention being that strings pronounced the same but spelled
    differently have identical encodings; words pronounced similarly
    should have similar encodings.
    """

    soundexcoding = [' ', ' ', ' ', ' ', ' ']
    soundexcodingindex = 1

    #           ABCDEFGHIJKLMNOPQRSTUVWXYZ
    mappings = "01230120022455012623010202"

    soundexcoding[0] = name[0].upper()

    for i in range(1, len(name)):
        c = ord(name[i].upper()) - 65
        if c >= 0 and c <= 25:

```

```

        if mappings[c] != '0':
            if mappings[c] != soundexcoding[soundexcodingindex-1]:
                soundexcoding[soundexcodingindex] = mappings[c]
                soundexcodingindex += 1
            if soundexcodingindex > 3:
                break

    if soundexcodingindex <= 3:
        while(soundexcodingindex <= 3):
            soundexcoding[soundexcodingindex] = '0'
            soundexcodingindex += 1

    return ''.join(soundexcoding)
soundex('EDUCATE'), soundex('EVALUATE')

```

[7]: ('E323', 'E143')

2.4 Similarity for Sets

For this exercise, you may use a tokenizing function that convert the following sentences into sets of words (words are separated by spaces). After that, apply Jaccard, Sørensen Coefficient, Tversky Index, Overlap Coefficient).

D1 = People wearing costumes are gathering in a forest and are looking in the same direction

D2 = People wearing costumes are scattering in a forest and are looking in different directions

D3 = People wearing costumes are gathering in a forest and are looking in the same direction

D4 = Masked people are looking in the same direction in a forest

Compute the pairwise similarity between the documents using the four measures.

```

[8]: '''
      TODO: first step is to tokenize the documents
      '''
      D1 = 'People wearing costumes are gathering in a forest and are looking in the_
            ↳same direction'
      D2 = 'People wearing costumes are scattering in a forest and are looking in_
            ↳different directions'
      D3 = 'People wearing costumes are gathering in the wood and are looking in the_
            ↳same direction'
      D4 = 'Masked people are looking in the same direction in a forest'
      S1 = D1.split()
      S2 = D2.split()
      S3 = D3.split()
      S4 = D4.split()

```

Jaccard Similarity

```

[9]: '''
      TODO: Compute the pairwise Jaccard similarity for the sets S1, S2, S3, and S4.

```

```
'''
def jaccard_similarity(set1, set2):
    intersection_size = len(set1.intersection(set2))
    union_size = len(set1.union(set2))
    return intersection_size / union_size if union_size != 0 else 0.0

strings = [S1, S2, S3, S4]
for i in range(len(strings)):
    s1 = set(strings[i])
    # loop over the columns of the second table
    s2 = set()
    for j in range(i+1, len(strings)):
        s2 = set(strings[j])
        JSim = jaccard_similarity(s1, s2)
        # display the Similarities
        print ("Jaccard similarity between S{} and S{} = {:.2.3}" \
              .format(i+1, j+1, JSim))
```

Jaccard similarity between S1 and S2 = 0.562
 Jaccard similarity between S1 and S3 = 0.786
 Jaccard similarity between S1 and S4 = 0.533
 Jaccard similarity between S2 and S3 = 0.412
 Jaccard similarity between S2 and S4 = 0.294
 Jaccard similarity between S3 and S4 = 0.375

Sørensen Coefficient

```
[10]: '''
TODO: Compute the pairwise Sørensen Coefficient for the sets S1, S2, S3, and S4.
'''
def Sørensen_Coefficient(set1, set2):
    intersection_size = len(set1.intersection(set2))
    total_size = (len(set1) + len(set2))
    return (2 * intersection_size) / total_size if total_size != 0 else 0.0

strings = [S1, S2, S3, S4]
for i in range(len(strings)):
    s1 = set(strings[i])
    # loop over the columns of the second table
    s2 = set()
    for j in range(i+1, len(strings)):
        s2 = set(strings[j])
        SCoef = Sørensen_Coefficient(s1, s2)
        # display the Similarities
        print ("Sørensen Coefficient between S{} and S{} = {:.2.3}" \
              .format(i+1, j+1, SCoef))
```

Sørensen Coefficient between S1 and S2 = 0.72
 Sørensen Coefficient between S1 and S3 = 0.88

Sørensen Coefficient between S1 and S4 = 0.696
Sørensen Coefficient between S2 and S3 = 0.583
Sørensen Coefficient between S2 and S4 = 0.455
Sørensen Coefficient between S3 and S4 = 0.545

Tversky Index

```
[11]: '''
      TODO: Compute the pairwise Tversky Index for the sets S1, S2, S3, and S4.
      '''
      def Tversky_Coefficient(set1, set2):
          intersection_size = len(set1.intersection(set2))
          S1_diff_S2 = len(set1.difference(set2))
          S2_diff_S1 = len(set2.difference(set1))
          alpha = 0.2
          beta = 0.8
          denominator = intersection_size + (alpha * S1_diff_S2) + (beta * S2_diff_S1)
          return intersection_size / denominator if denominator != 0 else 0.0

      strings = [S1, S2, S3, S4]
      for i in range(len(strings)):
          s1 = set(strings[i])
          # loop over the columns of the second table
          s2 = set()
          for j in range(i+1, len(strings)):
              s2 = set(strings[j])
              TCoef = Tversky_Coefficient(s1, s2)
              # display the Similarities
              print ("Tversky Coefficient between S{} and S{} = {:.2.3}" \
                    .format(i+1, j+1, TCoef))
```

Tversky Coefficient between S1 and S2 = 0.738
Tversky Coefficient between S1 and S3 = 0.902
Tversky Coefficient between S1 and S4 = 0.755
Tversky Coefficient between S2 and S3 = 0.583
Tversky Coefficient between S2 and S4 = 0.481
Tversky Coefficient between S3 and S4 = 0.577

Overlap Coefficient

```
[12]: '''
      TODO: Compute the pairwise Overlap Coefficient for the sets S1, S2, S3, and S4.
      '''
      def Overlap_Coefficient(set1, set2):
          intersection_size = len(set1.intersection(set2))
          min_size = min(len(set1), len(set2))
          return intersection_size / min_size if min_size != 0 else 0.0

      strings = [S1, S2, S3, S4]
```

```

for i in range(len(strings)):
    s1 = set(strings[i])
    # loop over the columns of the second table
    s2 = set()
    for j in range(i+1, len(strings)):
        s2 = set(strings[j])
        OCoef = Overlap_Coefficient(s1, s2)
        # display the Similarities
        print ("Overlap Coefficient between S{} and S{} = {:.2.3}" \
              .format(i+1, j+1, OCoef))

```

```

Overlap Coefficient between S1 and S2 = 0.75
Overlap Coefficient between S1 and S3 = 0.917
Overlap Coefficient between S1 and S4 = 0.8
Overlap Coefficient between S2 and S3 = 0.583
Overlap Coefficient between S2 and S4 = 0.5
Overlap Coefficient between S3 and S4 = 0.6

```

3 Locality Sensitive Hashing (LSH)

In the realm of large datasets and high-dimensional spaces, searching for items can be computationally challenging. Specifically, if you want to find items that are similar to a given item (known as the “query item”), conventional search methods might take an impractical amount of time. This is more observable in tasks that involve finding similar documents in a vast collection corpus or similar image in a massive image database.

Applying LSH for finding similar documents will require comparing the query with the documents in a specific bucket instead of comparing it with every single document in the collection. At its heart, LSH is a method for hashing data points in a way such that similar data points will, with high probability, end up in the same “bucket” or hash bin. Conversely, dissimilar items will, with high probability, end up in different bin

Unlike traditional hash functions used in dictionaries, which aim to reduce the likelihood of multiple key-values mapping to the same bucket, LSH operates differently. In dictionary hashing, minimizing collisions is crucial. On the contrary, LSH actively embraces collisions, especially for similar inputs. The essence of an LSH function is to group like values into identical buckets.

While the core principle of LSH is to bucket similar items using a hash function, the methodologies can differ significantly.

The technique we’re introducing, and which will be the point of this lab, is based on established methods involving shingling, MinHashing, and banding.

```

[13]: import requests
      from io import StringIO
      from IPython.display import display
      from random import shuffle
      from itertools import combinations

```

Loading data set

First we need data set. There are great repository in internet for testing similarity search testing. We will be extracting a set of sentences from [here](#).

```
[14]: # URL of the file
url = "https://raw.githubusercontent.com/brmson/dataset-sts/master/data/sts/
↳sick2014/SICK_train.txt"
response = requests.get(url)
if response.status_code == 200:
    content = response.text
    df_docs = pd.read_csv(StringIO(content), delimiter='\t')
    display(df_docs)
else:
    print("Failed to fetch data from the URL.")
```

	pair_ID	sentence_A \		sentence_B	relatedness_score \		entailment_judgment
0	1	A group of kids is playing in a yard and an ol...		A group of boys in a yard is playing and a man...	4.5		NEUTRAL
1	2	A group of children is playing in the house an...		A group of kids is playing in a yard and an ol...	3.2		NEUTRAL
2	3	The young boys are playing outdoors and the ma...		The kids are playing outdoors near a man with ...	4.7		ENTAILMENT
3	5	The kids are playing outdoors near a man with ...		A group of kids is playing in a yard and an ol...	3.4		NEUTRAL
4	9	The young boys are playing outdoors and the ma...		A group of kids is playing in a yard and an ol...	3.7		NEUTRAL
...	...	...		...	...		...
4495	9993	A door is being opened by a man		A bald man in a band is playing guitar in the ...	1.1		NEUTRAL
4496	9997	Someone is boiling okra in a pot		The man is not playing the drums	1.0		NEUTRAL
4497	9998	The man is singing heartily and playing the gu...		A bicyclist is holding a bike over his head in...	1.0		NEUTRAL
4498	9999	A man in blue has a yellow ball in the mitt		A man is jumping rope outside	1.2		NEUTRAL
4499	10000	Three dogs are resting on a sidewalk		The woman with a knife is slicing a pepper	1.0		NEUTRAL

4496	NEUTRAL
4497	NEUTRAL
4498	NEUTRAL
4499	NEUTRAL

[4500 rows x 5 columns]

3.1 Shingling, MinHashing, and LSH

The Locality-Sensitive Hashing (LSH) approach encompasses a three-step process. Initially, we convert text into sparse vectors using k-shingling and one-hot encoding. We then employ minhashing to generate ‘signatures,’ which are subsequently used in our LSH process to filter out potential candidate pairs. Finally, we group the signatures into buckets where each bucket contains the set of documents that are potentially similar. This will reduce the computational cost as the pairwise similarity will be performed only on the documents with signatures falling in the same bucket. If the number of buckets is large enough and the signatures are evenly distributed among the buckets, the computational cost will be reduced significantly.

3.1.1 k-Shingling

k-Shingling, often referred to as shingling, involves the transformation of a text string into a collection of ‘shingles.’ This procedure resembles sliding a window of length k along our text string and capturing a snapshot at each step. We aggregate all these snapshots to form our set of shingles.

```
[15]: def shingle(text: str, k: int)->set:
    """
    Create a set of 'shingles' from the input text using k-shingling.

    Parameters:
        text (str): The input text to be converted into shingles.
        k (int): The length of the shingles (substring size).

    Returns:
        set: A set containing the shingles extracted from the input text.
    """
    shingle_set = []
    for i in range(len(text) - k + 1):
        shingle_set.append(text[i:i+k])
    return set(shingle_set)
def build_vocab(shingle_sets: list)->dict:
    """
    Constructs a vocabulary dictionary from a list of shingle sets.

    This function takes a list of shingle sets and creates a unified vocabulary
    dictionary. Each unique shingle across all sets is assigned a unique integer
    identifier.

    Parameters:
```

- *shingle_sets* (list of set): A list containing sets of shingles.

Returns:

- *dict*: A vocabulary dictionary where keys are the unique shingles and values are their corresponding unique integer identifiers.

Example:

```
sets = [{"apple", "banana"}, {"banana", "cherry"}]
build_vocab(sets)
{'apple': 0, 'cherry': 1, 'banana': 2} # The exact order might vary due to set behavior

"""
full_set = {item for set_ in shingle_sets for item in set_}
vocab = {}
for i, shingle in enumerate(list(full_set)):
    vocab[shingle] = i
return vocab
def one_hot(shingles: set, vocab: dict):
    vec = np.zeros(len(vocab))
    for shingle in shingles:
        idx = vocab[shingle]
        vec[idx] = 1
    return vec
```

Simple example of shingling

```
[16]: '''
TODO: using the function 'shingle', find the list of 5-shingles in the document:
D = "This is an example of text shingling using k-shingling."
After that, find the 5-shingles manually and check if the computed ones are
correct or not.

If you don't see the message "Assertion failed: The result does not match the
expected shingles.",
then the function that computes the shingles provides correct results
'''

text = "This is an example of text shingling using k-shingling."
k = 5
result = shingle(text, k)
# We find all the shingles and use the set class to remove the duplicates
expected_result = {'This ', 'his i', 'is is', 's is ', ' is a', 'is an', 's an',
    ' an e', 'an ex',
    'n exa', ' exam', 'examp', 'xampl', 'ample', 'mple ', 'ple',
    'o', 'le of', 'e of '}
```

```

        ' of t', 'of te', 'f tex', ' text', 'text ', 'ext s', 'xt_
↪sh', 't shi', ' shin',
        'shing', 'hingl', 'ingli', 'nglin', 'gling', 'ling ', 'ing_
↪u', 'ng us', 'g usi',
        ' usin', 'using', 'sing ', 'ing k', 'ng k-', 'g k-s', '_
↪k-sh', 'k-shi', '-shin',
        'shing', 'hingl', 'ingli', 'nglin', 'gling', 'ling.', }
assert result == set(expected_result), "Assertion failed: The result does not_
↪match the expected shingles."

```

Use shingling on the set of document in the dataframe df_docs

```

[17]: '''
TODO:
    1. create a single list that contains all the sentences in sentence_A,
↪sentence_B
    2. find the list of 3-shingles from the full list of sentences
    3. Use the functions build_vocab and one_hot to create the_
↪documents-shinles matrix
    (you can also use your own functions)
'''
k = 3
sentences = df_docs['sentence_A'].tolist()
sentences.extend(df_docs['sentence_B'].tolist())
print(f"the number of sentences (documents):{len(sentences)}")
shingles = []
for sentence in sentences:
    shingles.append(shingle(sentence,k))
vocab = build_vocab(shingles)
print(f"Number of vocabulary is:{len(vocab)}")
shingles_1hot = []
for shingle_set in shingles:
    shingles_1hot.append(one_hot(shingle_set,vocab))
shingles_1hot = np.stack(shingles_1hot)
shingles_1hot

```

the number of sentences (documents):9000

Number of vocabulary is:3150

```

[17]: array([[1., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            ...,
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.]])

```

3.1.2 MinHashing

Minhashing represents the task of transforming the sparse vectors (in the documents-shingles matrix) into dense ones. Starting with the sparse vectors, for each slot in the signature, we randomly produce a single minhash function. For instance, if we aim to formulate a dense vector or signature with 20 values, we would deploy 20 hashing functions.

These hashing functions generate sequences of numbers arranged in randomly. We sequentially number them from 1 up to the number of shingles (vocab entries).

```
[18]: def get_minhash_arr(num_hashes:int,vocab:dict):  
    """  
    Generates a MinHash array for the given vocabulary.  
  
    This function creates an array where each row represents a hash function and  
    each column corresponds to a word in the vocabulary. The values are  
    ↪permutations  
    of integers representing the hashed value of each word for that particular  
    ↪hash function.  
  
    Parameters:  
    - num_hashes (int): The number of hash functions (rows) to generate for the  
    ↪MinHash array.  
    - vocab (dict): The vocabulary where keys are words and values can be any  
    ↪data  
      (only keys are used in this function).  
  
    Returns:  
    - np.ndarray: The generated MinHash array with `num_hashes` rows and  
    ↪columns equal  
      to the size of the vocabulary. Each cell contains the hashed value of the  
    ↪corresponding  
      word for the respective hash function.  
  
    Example:  
    vocab = {'apple': 1, 'banana': 2}  
    get_minhash_arr(2, vocab)  
    # Possible output:  
    # array([[1, 2],  
    #       [2, 1]])  
    """  
    length = len(vocab.keys())  
    arr = np.zeros((num_hashes,length))  
    for i in range(num_hashes):  
        permutation = np.random.permutation(len(vocab.keys())) + 1  
        arr[i,:] = permutation.copy()  
    return arr.astype(int)  
def get_signature(minhash:np.ndarray, vector:np.ndarray):
```

```

"""
    Computes the signature of a given vector using the provided MinHash matrix.

    The function finds the nonzero indices of the vector, extracts the
    corresponding
    columns from the MinHash matrix, and computes the signature as the minimum
    value
    across those columns for each row of the MinHash matrix.

    Parameters:
    - minhash (np.ndarray): The MinHash matrix where each column represents a
    shingle
    and each row represents a hash function.
    - vector (np.ndarray): A vector representing the presence (non-zero values)
    or
    absence (zero values) of shingles.

    Returns:
    - np.ndarray: The signature vector derived from the MinHash matrix for the
    provided vector.

    Example:
    minhash = np.array([[2, 3, 4], [5, 6, 7], [8, 9, 10]])
    vector = np.array([0, 1, 0])
    get_signature(minhash, vector)
    output: array([3, 6, 9])
    """
    idx = np.nonzero(vector)[0].tolist()
    shingles = minhash[:,idx]
    signature = np.min(shingles,axis=1)
    return signature

def jaccard_similarity(set1, set2):
    intersection_size = len(set1.intersection(set2))
    union_size = len(set1.union(set2))
    return intersection_size / union_size if union_size != 0 else 0.0

def compute_signature_similarity(signature_1, signature_2):
    """
    Calculate the similarity between two signature matrices using MinHash.

    Parameters:
    - signature_1: First signature matrix as a numpy array.
    - signature_matrix2: Second signature matrix as a numpy array.

    Returns:
    - Estimated Jaccard similarity.

```

```

"""
# Ensure the matrices have the same shape
if signature_1.shape != signature_2.shape:
    raise ValueError("Both signature matrices must have the same shape.")
# Count the number of rows where the two matrices agree
agreement_count = np.sum(signature_1 == signature_2)
# Calculate the similarity
similarity = agreement_count / signature_2.shape[0]

return similarity

```

```

[19]: '''
      TODO:
          1. create an array to hold 100 permutations (the second dimension will be
          ↳ the number of shingles for sure)
          2. create the signature matrix
             (the size of the matrix should be the number of permutations times the
          ↳ number of documents)
      '''

minhash_arr = get_minhash_arr(100,vocab)
signatures = []
for vector in shingles_1hot:
    signatures.append(get_signature(minhash_arr,vector))
signatures = np.stack(signatures)
signatures.shape

```

```
[19]: (9000, 100)
```

```

[20]: '''
      TODO: compute the simialrity between two documents using:
          1. Jaccard similarity on the documents-shingles arrays
          2. the signature based similarity using the function
          ↳ `compute_signature_similarity`
      '''

print(compute_signature_similarity(signatures[2],signatures[3]))
print(jaccard_similarity(shingles[3],shingles[2]))

```

```
0.45
```

```
0.42105263157894735
```

3.1.3 LSH Function

After preparing the data using the methods mentioned above, the LSH function is applied. This function divides the MinHash signature of each sentence into multiple bands and hashes each band. If two sentences have the same hash value for any band, they are considered to be potentially similar and are placed in the same bucket. Sentences within the same bucket are then checked more thoroughly to determine the degree of their similarity. By adjusting parameters like the

number of bands and rows, one can control the trade-off between precision and recall.

```
[21]: class LSH:
    """
    Implements the Locality Sensitive Hashing (LSH) technique for approximate
    nearest neighbor search.
    """
    buckets = []
    counter = 0

    def __init__(self, b: int):
        """
        Initializes the LSH instance with a specified number of bands.

        Parameters:
        - b (int): The number of bands to divide the signature into.
        """
        self.b = b
        for i in range(b):
            self.buckets.append({})

    def make_subvecs(self, signature: np.ndarray) -> np.ndarray:
        """
        Divides a given signature into subvectors based on the number of bands.

        Parameters:
        - signature (np.ndarray): The MinHash signature to be divided.

        Returns:
        - np.ndarray: A stacked array where each row is a subvector of the
        ↪signature.
        """
        l = len(signature)
        assert l % self.b == 0
        r = int(l / self.b)
        subvecs = []
        for i in range(0, l, r):
            subvecs.append(signature[i:i+r])
        return np.stack(subvecs)

    def add_hash(self, signature: np.ndarray):
        """
        Adds a signature to the appropriate LSH buckets based on its subvectors.

        Parameters:
        - signature (np.ndarray): The MinHash signature to be hashed and added.
        """
```

```

subvecs = self.make_subvecs(signature).astype(str)
for i, subvec in enumerate(subvecs):
    subvec = ','.join(subvec)
    if subvec not in self.buckets[i].keys():
        self.buckets[i][subvec] = []
    self.buckets[i][subvec].append(self.counter)
self.counter += 1

def check_candidates(self) -> set:
    """
    Identifies candidate pairs from the LSH buckets that could be potential
    ↪near duplicates.

    Returns:
    - set: A set of tuple pairs representing the indices of candidate
    ↪signatures.
    """
    candidates = []
    for bucket_band in self.buckets:
        keys = bucket_band.keys()
        for bucket in keys:
            hits = bucket_band[bucket]
            if len(hits) > 1:
                candidates.extend(combinations(hits, 2))
    return set(candidates)

```

Now, we create a set of *b* buckets to hold the potentially similar documents.

```

[22]: b = 10    # number of buckets
lsh = LSH(b)
for signature in signatures:
    lsh.add_hash(signature)
candidate_pairs = lsh.check_candidates()
len(candidate_pairs)

```

[22]: 17569

We display the top 5 similar candidates.

```

[23]: for candidate in list(candidate_pairs)[:5]:
    print("Candidate similar sentence are:")
    print(sentences[candidate[0]])
    print(sentences[candidate[1]])
    print("="*10)

```

Candidate similar sentence are:

A boy and a girl in swimsuits are wearing arm floats

A boy and a girl in swimsuits are wearing floats for their arms

=====

Candidate similar sentence are:

A woman is cutting an onion

A man is cutting onions

=====

Candidate similar sentence are:

A lone biker is jumping in the air

A lone biker is jumping in the air

=====

Candidate similar sentence are:

A shirtless man is playing football on a lawn

A shirtless man is playing football in a gym

=====

Candidate similar sentence are:

Little kids are playing in a water fountain in front of lots of people

Little boys are playing in a water fountain in front of lots of people

=====

```
[24]: '''  
      TODO: try displaying more documents that looks similar.  
      '''  
      number_of_candidates = 8  
      for candidate in list(candidate_pairs)[:number_of_candidates]:  
          print("Candidate similar documents are:")  
          print(sentences[candidate[0]])  
          print(sentences[candidate[1]])  
          print("="*10)
```

Candidate similar documents are:

A boy and a girl in swimsuits are wearing arm floats

A boy and a girl in swimsuits are wearing floats for their arms

=====

Candidate similar documents are:

A woman is cutting an onion

A man is cutting onions

=====

Candidate similar documents are:

A lone biker is jumping in the air

A lone biker is jumping in the air

=====

Candidate similar documents are:

A shirtless man is playing football on a lawn

A shirtless man is playing football in a gym

=====

Candidate similar documents are:

Little kids are playing in a water fountain in front of lots of people

Little boys are playing in a water fountain in front of lots of people

=====

Candidate similar documents are:

```

A man is playing the guitar
A man is playing the guitar
=====
Candidate similar documents are:
One woman is peeling a potato
A woman is peeling a potato
=====
Candidate similar documents are:
There is no man playing a guitar
There is no man playing the guitar
=====

```

```

[25]: from google.colab import drive
      drive.mount('/content/drive')

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```

[4]: %%capture
      !apt-get install -y texlive-xetex texlive-fonts-recommended texlive-latex-extra

```

```

[5]: %%capture
      !cp "/content/drive/MyDrive/Colab Notebooks/
        ↳String_similarity_heterogeneous_data.ipynb" ./

      !jupyter nbconvert --to latex "String_similarity_heterogeneous_data.ipynb"

      with open("String_similarity_heterogeneous_data.tex", "r", encoding="utf-8") as f:
        ↳f:
          tex = f.read()

      title_block = (
        r"\title{Heterogeneous data analysis and string similarity}" + "\n" +
        r"\author{Anthony Kamau}" + "\n" +
        r"\date{}" + "\n"
      )

      tex = tex.replace(r"\begin{document}", title_block + r"\begin{document}")

      if r"\maketitle" not in tex:
        tex = tex.replace(r"\begin{document}", r"\begin{document}" + "\n" +
        ↳r"\maketitle", 1)

      with open("String_similarity_heterogeneous_data.tex", "w", encoding="utf-8") as f:
        ↳f:
          f.write(tex)

      !xelatex String_similarity_heterogeneous_data.tex

```

```
!xelatex String_similarity_heterogeneous_data.tex
```

```
[6]: from google.colab import files  
files.download("String_similarity_heterogeneous_data.pdf")
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>